CEUE203 - OOP

PRACTICAL – 8

Name: Jay
ID No.: 25AIML034
Subject: CEUE203 - OOP
Practical: 8

Aim:

To implement custom exceptions and structured exception handling in Java for a fault-tolerant MiniBank system.

Part A – Practice Programs

1. Guarded Calculator

Read two numbers and an operator, handle invalid input separately, use a custom DivideByZeroException, and continue until a valid calculation succeeds.

```
import java.util.*;

class DivideByZeroException extends Exception {
    public DivideByZeroException(String message) {
        super(message);
    }
}

public class GuardedCalculator {
    static double calculate(double a, double b, char op)
            throws DivideByZeroException {
        if (op == '/' && b == 0)
            throw new DivideByZeroException("Cannot divide by zero");

        return switch (op) {
            case '+' -> a + b;
            case '-' -> a - b;
            case '*' -> a * b;
            case '/' -> a / b;
            default -> throw new IllegalArgumentException("Invalid operator");
        };
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        boolean success = false;

        while (!success) {
            try {
                double a = sc.nextDouble();
                double b = sc.nextDouble();
```

```
                char op = sc.next().charAt(0);

                System.out.println("Result: " + calculate(a, b, op));
                success = true;
            } catch (InputMismatchException e) {
                System.out.println("Invalid number input");
                sc.nextLine();
            } catch (DivideByZeroException e) {
                System.out.println(e.getMessage());
                sc.nextLine();
            } finally {
                System.out.println("Attempt completed");
            }
        }
    }
}
```

2. Stock Issue

Process warehouse issue requests and report invalid quantities or stock shortages without stopping the complete run.

```
class OutOfStockException extends Exception {
    private int shortfall;

    public OutOfStockException(int shortfall) {
        super("Out of stock");
        this.shortfall = shortfall;
    }

    public int getShortfall() {
        return shortfall;
    }
}

class InvalidQuantityException extends Exception {
    public InvalidQuantityException(String message) {
        super(message);
    }
}

class Warehouse {
    private int stock = 20;

    public void issue(String item, int qty)
            throws OutOfStockException, InvalidQuantityException {
        if (qty <= 0)
            throw new InvalidQuantityException("Quantity must be positive");

        if (qty > stock)
            throw new OutOfStockException(qty - stock);

        stock -= qty;
        System.out.println(item + " issued: " + qty);
    }
}

public class StockIssue {
    public static void main(String[] args) {
        Warehouse w = new Warehouse();
```

```
        int[][] requests = {{5}, {30}, {0}, {10}};

        for (int[] request : requests) {
            try {
                w.issue("Item", request[0]);
            } catch (OutOfStockException e) {
                System.out.println("Shortfall: " + e.getShortfall());
            } catch (InvalidQuantityException e) {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

3. Advanced Learner – AutoCloseable

Try-with-resources closes the resource automatically even when an exception occurs inside the block.

```
class BankResource implements AutoCloseable {
    public void open() {
        System.out.println("Resource opened");
    }

    @Override
    public void close() {
        System.out.println("Resource closed");
    }
}

public class ResourceDemo {
    public static void main(String[] args) {
        try (BankResource resource = new BankResource()) {
            resource.open();
            throw new RuntimeException("Original error");
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Part B – Project Work: MiniBank

1. Exception Hierarchy

```
package exception;

public class BankException extends Exception {
    public BankException(String message) {
        super(message);
    }
}

public class InsufficientFundsException extends BankException {
    private long shortfall;

    public InsufficientFundsException(long shortfall) {
        super("Insufficient funds");
        this.shortfall = shortfall;
    }
```

```
    public long getShortfall() {
        return shortfall;
    }
}

public class AccountNotFoundException extends BankException {
    public AccountNotFoundException(String message) {
        super(message);
    }
}

public class InvalidAmountException extends BankException {
    public InvalidAmountException(String message) {
        super(message);
    }
}
```

2. Account Operations

```
import exception.*;

public class Account {
    private long balance;

    public Account(long balance) {
        this.balance = balance;
    }

    public void withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException {
        if (amount <= 0)
            throw new InvalidAmountException("Amount must be positive");

        if (amount > balance)
            throw new InsufficientFundsException(amount - balance);

        balance -= amount;
    }

    public void deposit(long amount)
            throws InvalidAmountException {
        if (amount <= 0)
            throw new InvalidAmountException("Amount must be positive");

        balance += amount;
    }

    public void transfer(Account to, long amount)
            throws BankException {
        try {
            withdraw(amount);
            to.deposit(amount);
        } catch (BankException e) {
            throw e;
        } finally {
            System.out.println("Transfer attempt completed");
        }
    }
}
```

3. Main Program

```
    public class Main {
        public static void main(String[] args) {
            Account account = new Account(1000);

            try {
                account.withdraw(5000);
            } catch (InsufficientFundsException e) {
                System.out.println("Insufficient funds");
                System.out.println("Shortfall: " + e.getShortfall());
            } catch (InvalidAmountException e) {
                System.out.println(e.getMessage());
            } finally {
                System.out.println("Operation completed");
            }

            try (BankResource resource = new BankResource()) {
                resource.open();
            } catch (Exception e) {
                System.out.println(e.getMessage());
            }
        }
    }
```

Expected Output:

```
    Insufficient funds
    Shortfall: 4000
    Operation completed
    Resource opened
    Resource closed
```

Code Screenshot

```
Account account = new Account(1000);

try {
    account.withdraw(5000);
} catch (InsufficientFundsException e) {
    System.out.println("Insufficient funds");
    System.out.println("Shortfall: " + e.getShortfall());
} finally {
    System.out.println("Operation completed");
}
```

Output Screenshot

```
Insufficient funds
Shortfall: 4000
Operation completed
```

Key Questions

1. What is the difference between a checked and an unchecked exception?

Checked exceptions are checked by the compiler and must be handled or declared. Unchecked exceptions extend RuntimeException and are not required to be declared.

2. What is the purpose of the finally block, and when does it run?

The finally block is used for cleanup and normally runs whether an exception occurs or not.

3. How does try-with-resources guarantee a resource is closed?

A resource implementing AutoCloseable is automatically closed at the end of the try block, even when an exception occurs.

Learning Outcome:

The student can build fault-tolerant Java programs using custom checked exceptions, throw and throws, try-catch-finally, exception propagation, and try-with-resources.

Post Laboratory Work:

Commit and tag lab-08. Confirm that withdrawing more than the account balance throws InsufficientFundsException and is caught with a clear message.